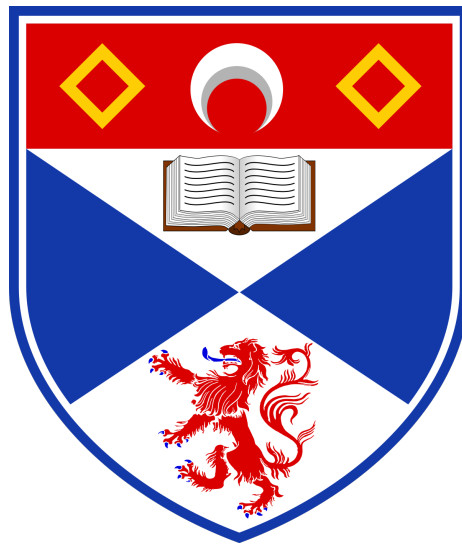


Interlinear text on web pages

Deepankur John Njondimackal

210002191



Project Supervisor: Dr Mark-Jan Nederhof

School of Computer Science
University of St Andrews
March 25, 2024

Declaration

I declare that the material submitted for assessment is my own work except where credit is explicitly given to others by citation or acknowledgement. This work was performed during the current academic year except where otherwise stated.

The main text of this project report is 9341 words long, including project specification and plan.

In submitting this project report to the University of St Andrews, I give permission for it to be made available for use in accordance with the regulations of the University Library. I also give permission for the title and abstract to be published and for copies of the report to be made and supplied at cost to any bona fide library or research worker, and to be made available on the World Wide Web. I retain the copyright in this work.

I also would like to thank my family, who have been incredibly loving these past three years of my university education and a reliable bedrock in both joyous and troubled times. I would not dream of making it this far without them.

Dispute of a man with his soul
Egyptian text from the Middle
Kingdom

Abstract

This paper describes the design and implementation of a web application called PhilologEg-Web that displays interlinear text, with a focus on Ancient Egyptian hieroglyphic texts. The application organizes texts from a corpus into categories like Books, Kings, and Museums. For each text, it displays multiple tiers such as the hieroglyphics, transliteration, and translation, allowing the user to adjust parameters like font size and hieroglyph scaling. A key challenge was developing algorithms to properly align and wrap the different tiers, particularly dealing with the two-dimensional and variable-length nature of hieroglyphic writing compared to linear scripts. The paper outlines the approaches taken, including segmenting the Latin tiers based on width, leveraging previous tier positions to initially place hieroglyphs, breaking hieroglyph fragments when overflowing, and redistributing hieroglyphs to fill gaps. The application was built using Node.js, Express, Jade templating, and leveraged prior work on RES (Revised Encoding System) for hieroglyph manipulation and rendering. An evaluation showed the primary objective of displaying interlinear hieroglyphic texts was achieved satisfactorily, while noting several potential areas for future work like unlimited tiers, orthographic visualization, editing functionality, and improved user interactions.

CONTENTS

Contents

1	Introduction	6
1.1	Background	6
1.2	Objectives	6
2	Literature Review	8
2.1	Interlinear Text	8
2.2	Egyptian Hieroglyphics	10
3	Design	13
3.1	Overview	13
3.2	Algorithms	15
3.2.1	Line Segmentation for Latin Script Tiers	16
3.2.2	RES Manipulation	18
3.2.3	Hieroglyphic Tier Breaking	19
3.3	Ordering	23
4	Implementation	24
4.1	Technologies	24
4.2	Navigation	24
4.3	Application Stack	24
4.4	Customizability	25
4.5	Caching	25
4.6	Anchor Alignment	26
4.7	RES Insertion	27
4.8	Conversion from RES to Hieroglyphics	27
4.9	Conversion from Canvas to SVG	28
4.10	Gap Filling	29
4.11	Printing	29
5	Evaluation	30
6	Future Work	31
7	Conclusion	32
8	References	33
8.1	Research Literature	33
8.2	Egyptological Texts	33

1 Introduction

1.1 Background

Interlinear text is the display of multiple tiers of writing fragments that are linked to each other. Within this project we will be looking at this concept as it relates to the display of Ancient Egyptian texts. These often have at least three tiers, i.e. the Hieroglyphics, the transliteration and the translation, which can be further increased by the presence of competing translations in the same language or translations into multiple languages. A unique niche exists in relation to Ancient Egyptian because of the nature of hieroglyphics introducing challenges around alignment of glyphs in a more two-dimensional space compared to the Latin script.

A previous project, *Automatic Creation of Interlinear Text for Philological Purposes* (Nederhof), demonstrated a proof-of-concept for loading a corpus of multiple Egyptian texts, displaying them accurately and organizing them based on academic attributes. This project featured the RES system pioneered by Nederhof for the purpose of encoding hieroglyphs in a textual format and was engineered as a Java application, which is no longer compatible on web browsers. It is crucial for us therefore to investigate the reconstruction of this project using modern technologies and take a look at avenues for further functionality. We will also have to re-evaluate some core design decisions, since the standard for hieroglyph representation has advanced significantly since the original project, particularly with Unicode. The undertaking to create a coherent web application for the organization and display of Ancient Egyptian archives also introduces fresh considerations, particularly around adapting to variable display sizes, such as shifting between a widescreen monitor and a smartphone screen.

1.2 Objectives

Primary

Implement the PhilologEg application from the aforementioned project using JavaScript. This application would read the corpus metadata from XML and organize the texts according to categories like Books, Kings and Museums. It would also display multiple tiers within each text. It would demonstrate adaptability to variance in display screen sizes.

INTRODUCTION — OBJECTIVE

Secondary

Provide an API service to accept text information and produce line alignment, building on the algorithm discussed in the aforementioned paper. Such an API could be generalized to work with writing systems other than just Egyptian Hieroglyphics such as cuneiform or traditional Mongolian. Extend the application to provide annotation for images and 3D scans such as hieroglyphic inscriptions on sarcophaguses. Provide full editing functionality for the texts that encompasses all the tiers involved.

Tertiary

Collect user feedback from a Human Computer Interaction perspective regarding the application.

2 Literature Review

In order to understand the context of the project, it is important to look at two main categories of study. The first would be the existing literature on the topic of interlinear text, and the second would be the existing literature on the topic of Egyptian Hieroglyphics. Obviously the work that we do within this project has a more universal scope in mind for the long-term where the approaches and methods developed are applicable not just to Egyptian hieroglyphics but other unconventional writing systems, and ideally, any writing system. That being said, it was still necessary for a particular writing system to be chosen as the primary proof-of-concept for our application and Egyptian Hieroglyphics was an obvious choice due to the large corpus available and prior work in interlinear rendering done specifically at the University of St Andrews. And although the core of the project would be agnostic towards the nature of the tier, when incorporating a particular writing system, designing algorithms specific to the writing systems' nuances and needs is still essential, which is where the prior body of research regarding Egyptian Hieroglyphics becomes useful.

2.1 Interlinear Text

A very useful source regarding the problem of Interlinear Text representation in an online web context was Schroeter et al.'s paper from 2006 regarding methods for online representation of interlinear text. Their work is interesting particularly because it represents both the diversity of approaches and common trends in solving this problem — their work was with Oceanic languages like South Efate and Aboriginal Australian languages, rather than any ancient ideographic writing system. However, they also made use of XML formats as the backbone for representing interlinear text information. Their systems, such as the EOPAS format, offer a high degree of depth in terms of annotation, going down four levels from the text level to phrase, word and even individual morphemes. Since their working context is speech transcription, they also include millisecond-specific timestamps as anchors. The result, when deployed to the web in the EthnoER system, is a highly information-dense view with a high degree of ease of navigation, as shown here. However, their system is no longer online, as is not surprising given the age of the paper, and so it

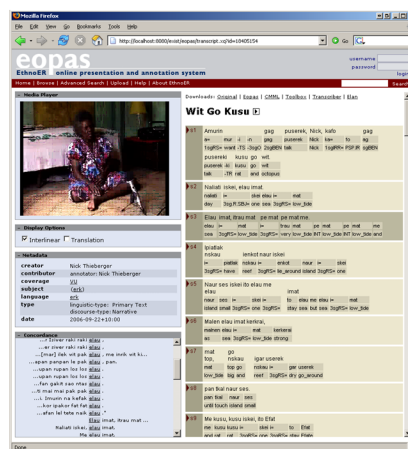


Figure 2.1: Schroeter et al.'s implementation

is difficult to ascertain how well it would have adapted to modern web technologies. Of particular note is the lack of responsiveness to different screen sizes, which is a key feature of our project. In their implementation, text is wrapped within the existing lines, instead of any attempt to redistribute and create new lines.

Nevertheless, the work of Schroeter et al. proffers key learning points for our own work. They identify two notable challenges in their own work at adapting interlinear text for the web — firstly, the lack of support for non-standard characters such as seen with the word “natamñol” in South Efate getting split into “natam” and “ol” due to the use of a combining tilde that is not recognised by their tokeniser; and thus the need to be careful with the font selection available due to the limited amount of support for special characters common in certain languages.

This problem is very relevant to Egyptian Hieroglyphics, which exhibit a high degree of non-standard control characters like ligatures and diacritics used to achieve correct display. In a web context, this limits our ability to have hieroglyphics displayed as “pure text” the same way Latin scripts or indeed most modern languages such as Korean or Hindi can be. For web applications, indexability by search engines and general accessibility of information by users is a core design paradigm, yet this is a goal that we still struggle with when it comes to Egyptian Hieroglyphics. The current approaches towards rendering Hieroglyphics for the web involve either Unicode within SVGs or using a Canvas to draw the glyphs, with the former being difficult to use for internal manipulation and second being inaccessible as proper text.

The font question has been a notable concern for the PhilologEg software in general even preceding the current work towards deployment on the web. On the one hand, we would like to offer users as much customizability as possible. On the other hand, even for something like Ancient Egyptian transliteration, we have to balance the need for a high proportion of diacritical marks with the fact of easy information storage in XML. This has limited our options for fonts in that context to ones like Trlit_CG Times 2023 from the *13th International Congress of Egyptologists, Leiden*, and in the case of the hieroglyphs to NewGardiner or modified Aegyptus from Nederhof. At least in the case of transliteration however, there has been some progress towards proper use of Unicode to make the text font-agnostic, as seen with an online converter¹ that could plausibly be incorporated in the workflow of our own project. A more in-depth consideration of this issue, and others linked to it can be found in the Implementation section.

¹<https://pnm.uni-mainz.de/tools/unicode/>; Alexander Ilin-Tomich, Institut für Altertumswissenschaften, Ägyptologie

The second challenge identified by Schroeter et al. is the wrapping of interlinear text, which they call a “notorious issue” with reference to previous work by Schmidt, Bow, Hughes and Bird. Their solution, which they considered sub-par, was to wrap within the table cells that they used for the view. We believe that this would not be the best option for encouraging interlinear readability, and thus would prefer to pursue solutions that redistribute the text in cases of overflow.

Other literature did give more attention to the issue of breaking and wrapping lines in interlinear texts to adapt to different widths.

<i>naḏa-ḡi</i> then-LOC	<i>kaṭi</i> clothing-ABS	<i>ḡuḡara-ḡḡa</i> take off-PART
<i>puḏ-ḡi</i> AUX-PRES	<i>ḡḡu</i> naked	<i>waḡḡa-ḡḡa</i> go-PART
<i>puḏ-ḡai</i> AUX-IMPL ₂	<i>ḡaḡi</i> meat	<i>ḡanka-ḡi</i> rotten-ERG
<i>wiḡḡa</i> paint-PART	<i>puḏ-ḡa</i> AUX-REL ₂	

Then they took their clothes off to walk around naked painted with some rotten meat

<i>naḏa-ḡi</i> then-LOC	<i>kaṭi</i> clothing-ABS	<i>ḡuḡara-ḡḡa</i> take off-PART	<i>puḏ-ḡi</i> AUX-PRES	<i>ḡḡu</i> naked	<i>waḡḡa-ḡḡa</i> go-PART
<i>puḏ-ḡai</i> AUX-IMPL ₂	<i>ḡaḡi</i> meat	<i>ḡanka-ḡi</i> rotten-ERG	<i>wiḡḡa</i> paint-PART	<i>puḏ-ḡa</i> AUX-REL ₂	

Then they took their clothes off to walk around naked painted with some rotten meat

Figure 2.2: Bow et al.’s interlinear text

while maintaining more conservative constraints for the interlinear gloss. In their case however, they have more granular alignment information going down to the morpheme level, whereas for us we have mainly vertical anchors and precedence information to work with. Nevertheless, this is a useful insight for our own work, as we will have to consider the nature of the different tiers in our interlinear texts and how they should be broken and wrapped. The authors also note that the wrapping of interlinear text is a complex problem, and that the solution they propose is not perfect, but rather a good starting point for further work. This is a sentiment that we can echo in our own work, as we will have to make a number of decisions about how to break and wrap the different tiers in our interlinear texts, and that these decisions will necessarily be compromises between competing concerns.

2.2 Egyptian Hieroglyphics

For a good understanding of the nuances of Egyptian Hieroglyphics, we mainly referred to the first chapter of Wildish’s 2012 doctoral thesis. While the paper as a whole is a comprehensive analysis of the hieroglyphic writing system in relation to late Platonist metaphysics, it provides an excellent overview of Egyptian Hieroglyphics where concepts proved useful in the Design of our project.

Wildish explains that the main body of written Ancient Egyptian studied is from the Middle Egyptian variant dating 2240 B.C. to 1990 B.C.. Texts can appear in all manner of orientations, from a choice of left-to-right, right-to-left or top-to-bottom, and the direction of the text is indicated by the direction the glyphs are facing.

Similar to Hebrew and Arabic, fellow members of the Semitic language family, Ancient Egyptian writing is unpointed, or in other words, lacking vowels. The script is a combination of logograms, representing words or morphemes, and phonetic glyphs, representing sounds. The logograms can be further divided into ideograms, which represent a single word, and determinatives, which are used to clarify the meaning of the preceding word. The phonetic glyphs can be further divided into unilaterals, which represent a single consonant, and bilaterals and triliterals, which represent two or three consonants respectively. The script is also known for its use of ligatures, where glyphs are combined into a single glyph, and for its use of determinatives, which are used to clarify the meaning of the preceding word.

Crucially, Wildish informs us that the distinction between sound-signs, which convey their sound rather than meaning, and sense-signs, which convey their meaning rather than their sound, has significant ramifications for the transliteration and translation of Ancient Egyptian texts, notably making it a one-way process. The transliteration for instance, can be thought of as compressing the information in the original Hieroglyphs in a sense, as can be seen quite evidently in Figure 3.4 in the Design section. In contrast, the translation into a modern language is an interpretative process that differs widely based on the perspective of the translator, and often involving “joining the dots” and generally creating a longer text than the original. A good illustration of this inconsistency can be seen in this Figure from Nederhof’s 2009 paper.

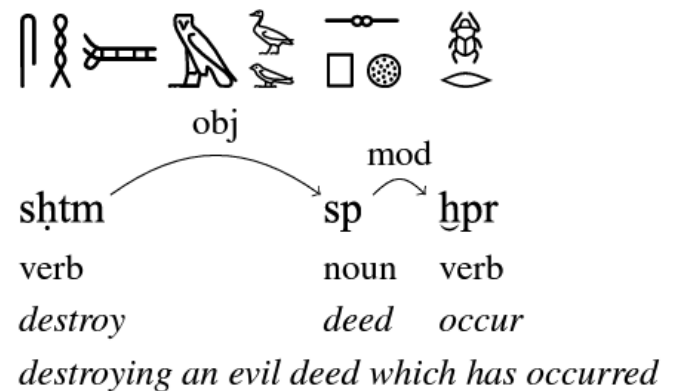


Figure 2.3: Variable Content Size

What all of this entails is that the relationship between the different tiers in a typical Ancient Egyptian interlinear text is not a simple one-to-one relationship, but a many-to-many relationship. Unlike translation between modern languages, there cannot be

LITERATURE REVIEW — EGYPTIAN HIEROGLPHICS

some general consistency of ratio for example, to be expected between the length of the Hieroglyphic tier and its translation or transliteration. This greatly limits the methods available to us for obtaining information around which to make alignment decisions between different tiers. Any solution would necessarily have to be creative at overcoming these hurdles to provide a satisfactory alignment of tiers.

3 Design

3.1 Overview

The overall structure of the application closely follows the original Java package by Nederhof. The home view of the application will display the list of texts, as well as different pages for lists of categories, such as Books, Kings and Museums. Clicking on a category will display a list of texts within that category. Clicking on a text will display the interlinear text in its own view. The application supports the loading of different corpuses, as long as they follow a standardised XML format for their index, and by default it uses the St Andrews Egyptology corpus.

Crucially the application is designed to be responsive to different screen sizes, so that it can be used on a smartphone or a tablet as well as a desktop computer, as seen in Figure 5.1 in the Evaluation section. It also allows the user to adjust the font size of the Latin tiers, and the relative scale of the hieroglyphs. Each combination of Screen Width, Font Size and Hieroglyph Scale can be considered a unique “view” of the text, and the application will have to reload the text and come up with a new solution for interlinear display each time the user changes any of these parameters. The results are then cached locally so that they can be loaded quickly the next time.

As can be seen in Figure 3.2, the web application has an open and clean appearance in keeping with the current common practices in web design. As much of the interface as possible is made directly accessible to the user instead of being hidden behind menus or tabs. Different tiers are also shaded in contrasting colours so that they are easily distinguishable from each other, which is particularly important when scrolling through larger texts. Other than navigating through the links on the application, each page has its own URL, so that the user can bookmark or share a specific text or category.

Discussion of more technical details, such as those relating to the front-end and back-end frameworks, will be conducted in the Implementation section. Within this section however, we hope to focus less on the web development side of the project — which was quite substantive in its own right — and more on the decisions made in devising the various algorithms for arranging and aligning interlinear text.

DESIGN — EVOLUTION OF APPLICATION

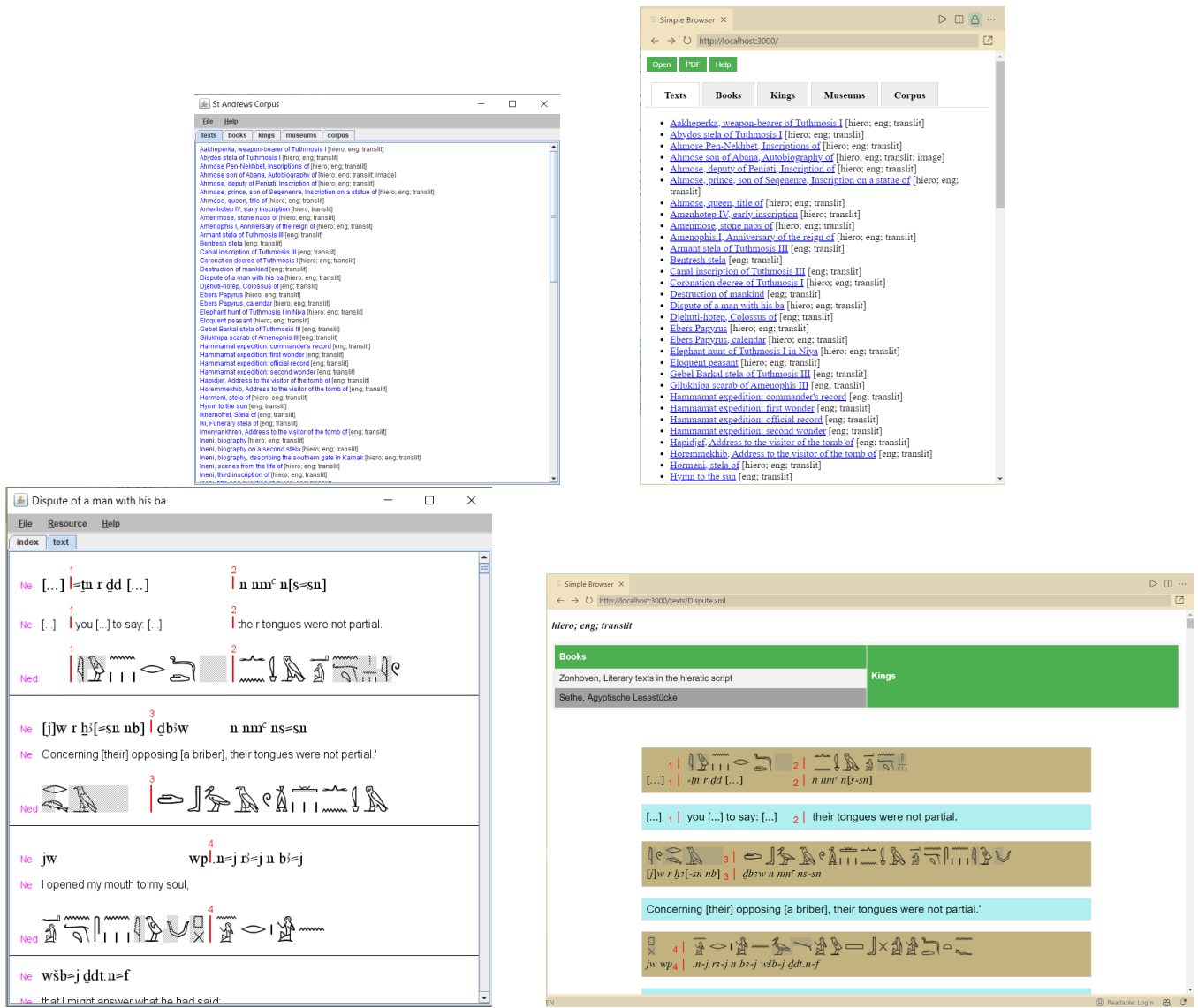


Figure 3.1: Comparison of the original and new design

3.2 Algorithms

The application follows a highly modular design, where different tasks relating to different tiers are separated as much as possible, so that there is little interference between them, and so that the solution is more generalizeable and more extendable for future work, such as working with other writing scripts.

It should be noted that there is no single perfect answer for how to display interlinear text. Since the translation into a different language is never a 1:1 conversion, there are multiple possibilities for what could be considered an acceptable arrangement, which is particularly complicated when we consider that the hieroglyphic script is not a purely linear script, but a two-dimensional one. The application therefore has to make a number of decisions about how to break the tiers, by balancing the competing concerns of aesthetics, accessibility, and accuracy.

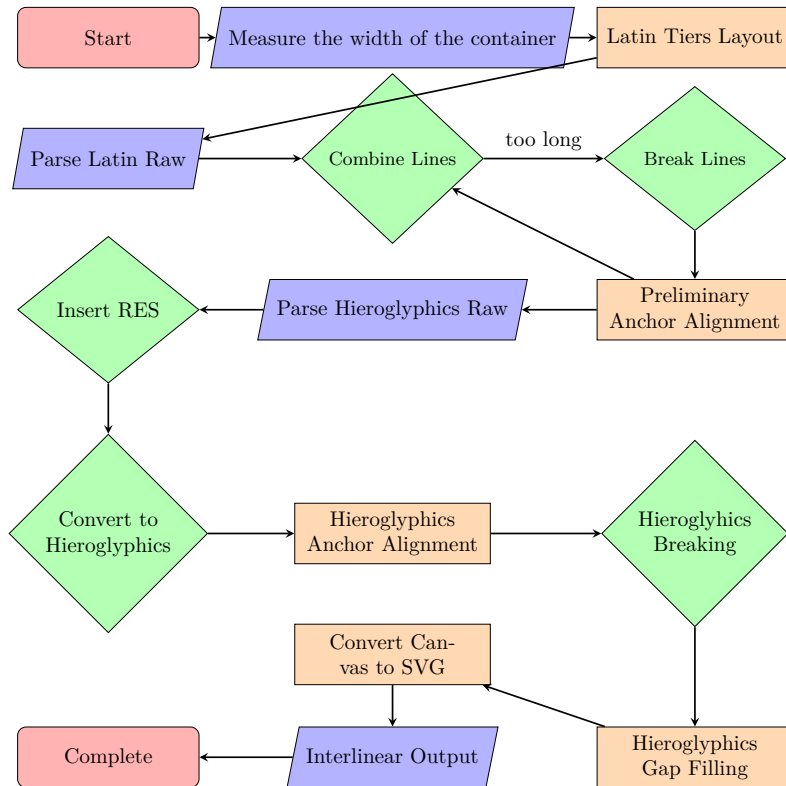


Figure 3.2: Interlinear Text Workflow

The tier manipulation methods were not lifted wholesale from those used in the original Java application, since the extensive object oriented facilities of Java meant that a much more formalised algorithm for tier breaking and alignment could be implemented. In the context of our web application though, we generally pursued a much more procedural approach to the problem, which was more in keeping with the nature of JavaScript and the DOM. We “solve” the problem of rendering the interlinear text in stages, with each function focusing on one particular sub-task. The algorithm for breaking the tiers was also made more general, so that it could be used for any writing system, and not just Egyptian Hieroglyphics.

3.2.1 Line Segmentation for Latin Script Tiers

Broadly speaking, when we consider the problem of segmenting the Latin script tiers of an interlinear text, we realise that it is a considerably more straightforward problem than the one we face with the hieroglyphic script. The Latin script is a linear script, and so we can simply break the tiers into lines based on the width of the container, and then align the lines with each other.

Complications arise if we consider the fact that Latin characters are of variable widths, and that the original interlinear text viewer allowed for the possibility of picking and choosing font families as well as font sizes. This means that a perfect solution would necessarily calculate the width of Latin script fragments by summing up the individual widths of each character within them. Such an approach is inescapable for the purposes of hieroglyphs, which we will discuss later. This is a computationally expensive operation, and so we decided to take a more heuristic approach for the Latin scripts, where we would estimate the average width of Latin characters based on just the font size and use this to obtain a maximum number of characters for each line — with an fine-tuned buffer to prevent overflow — and then work on assigning fragments to lines.

We use a greedy approach for creating the Latin script lines, where fragments are combined into a single line until the limit is exceeded. The overflow is then broken off into its own fragment and is prepended to the following line. This process is repeated until all the fragments are assigned to lines. Obviously, we want to avoid cutting words in the middle as far as possible and maintain semantic proportionality between different lines. Hence we use a concise penalty system to determine the best place to break the lines, by having a table for each type of tier (currently just Egyptological script and English translation) that sets certain characters as being more or less desirable to break on.

Character	Penalty	Scaled to Position
Space preceding an Align Anchor	−30	No
Space	−25	Yes
Period, Exclamation Mark or Question Mark	−20	No
Quotation Marks after a Period, Exclamation Mark or Question Mark	−15	No
Comma, Semi-colon or Colon	−10	No
Others	−5	Yes

Table 3.1: English Tier Penalty System

Crucially, the above system is independent of any other tiers and ensures a balanced distribution of the tier content across lines. It does not include information about the meaning of the texts or attempt to link fractions of fragments with proportionate fractions of fragments in other tiers. This is a deliberate decision, since the application is designed to be as general as possible and to work with any writing system, and not just Egyptian Hieroglyphics. The application is also designed to be as fast as possible, and so we have to make a trade-off between the accuracy of the line breaking and the speed of the application. The penalty system is therefore designed to be as simple as possible, and to be as fast as possible to calculate.

There is one exception to the above statement, and that is the relationship of the Latin Tiers with each other during segmentation. This is represented in Figure 2 by the fact that Preliminary Anchor Alignment occurs frequently in-between line segmentation of Latin Tiers. Logically this makes sense, even though the breaking decision for one of the Latin tiers does not incorporate information about another Latin tiers, at the end of the day they are still linked to each other through the vertical anchor markers, and an interlinear text solution would have anchors lining up with each other as a minimum requirement for success. The act of aligning the anchors of one tier with another can cause overflow on a line where it did not previously occur, hence it makes sense that for example, if the segment we choose to break off contains an anchor with a counterpart in other Latin tiers, the other tiers' lines should also be broken at that anchor.

3.2.2 RES Manipulation

One of the most critical components of the application is the RES system, which is used to encode the hieroglyphics in a textual format. As mentioned previously in the Literature Review, it was developed by Nederhof based on the Manuel de Codage system, and assigns alphanumeric shortcodes for specific glyphs.

Beyond its utility as a medium for storage of the hieroglyphic tiers of our interlinear texts however, RES is also useful as an internal format for the application to keep track of different fragments while making decisions on breaking them apart or joining them. When it comes to the actual rendering of Hieroglyphics, or indeed any non-Latin script, there are a variety of valid options available, from putting the Unicode text directly on the website to drawing them on HTML Canvases or constructing SVGs. Our choice and its reasoning is elaborated on in the Implementation section, however this is something that can change alongside evolving trends in web technologies and common practices, and our overarching design should not beholden to these choices, and should in fact be flexible in adapting to different implementation choices for rendering.

Given a particular fragment of RES, we need to be able to firstly be able to determine what its suitable breaking points are, and the width of sub-fragments obtained from acting upon these breakpoints. This is handled using a library previously developed by Nederhof called *ResTierPart* which takes in these two parameters and produces all of this information. For example, if we consider the RES fragment `m-Htp:t-p:Y1-tA:N23*Z1-n:Z2-pH:D54-n:Z2-sw-W` (where the usual beginning control sequences have been omitted for brevity), we can see that it can correlates into the following hieroglyphic sequence as represented orthographically in Figure 3. Of the thirty-one leaf nodes in this tree, seven are ligature controls (* or : in RES) which cannot be used as breaking points, since it would lead to nonsensical fragments being produced, or the meaning of the text being corrupted. This leaves seventeen possible breaking points, individual glyphs and the spaces between them, which we can choose from while creating sub-fragments.

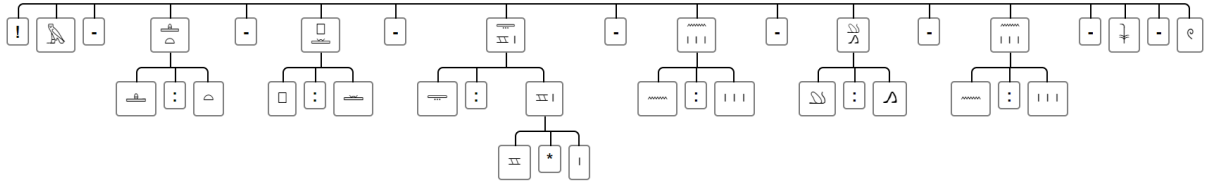


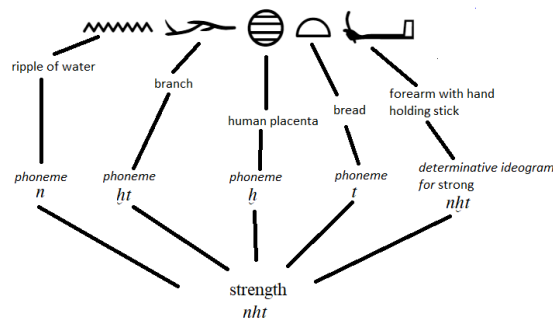
Figure 3.3: Hieroglyphic Fragment Structure

Next, we need the actual width of the sub-fragments. This is a more complicated problem than it might seem at first, since the width of a hieroglyphic fragment is not simply the sum of the widths of its constituent glyphs. The width of a fragment is also affected by the spaces between them, the ligature controls and the font family. If hieroglyphics were a linear script, we could query the font for the individual advance widths (which includes the spacing between glyphs) and sum them. However as seen in Figure 3, ligatures that combine glyphs vertically are reasonably common, which leads to the original glyphs being shrunk and the actual advance width being changed. Hence, the *ResTierPart* library deals with this by using an internal HTML Canvas to draw out the actual glyph and count its pixels. Within the context of our wider application of course, these design choices are abstracted away, and the application simply calls the library and receives the width of the sub-fragments.

With both of these features in place, we are able to use the *ResTierPart* library to conduct breaks at valid breaking points and obtain the corresponding RES representations of each fragment along with their widths. This equips us to conduct the Hieroglyphic tier fragmentation, as required while displaying it in the interlinear text viewer.

3.2.3 Hieroglyphic Tier Breaking

There were a few different options for how Hieroglyphic Tiers could have been fragmented, and the choice of the best one was not straightforward. We could have used an identical approach as the Latin Script, where we attempt to stuff as many fragments as possible into a line until the limit is hit, before breaking it off and repeating to redistribute for the entire text. However, one key issue with this was that unlike Latin script tiers, especially the translation tiers, there are a lot less natural breaking points in the Hieroglyphic script. We do not really have punctuation or word boundaries to guide us, which means that this method would likely result in a semantically less satisfying distribution of the text across lines. By way of example, we can consider this orthographic visualization we created based on an example from Mark Widdish’s 2012 paper.



Here it is quite apparent that despite comprising five glyphs, the entire fragment is a single word meaning “strength”. However, each glyph also has an individual meaning, and although most of them in this case are being used as phonemes, these same glyphs could in other contexts

Figure 3.4: Orthographic Visualization

be ideograms, i.e. each glyph would correspond to a single word. In other words, the density of meaning encoded by hieroglyphics is highly variable even within a single text. So if a penalty system similar to the one we used for the Latin script tiers is untenable, what other options do we have on the table?

Another approach that offers itself to our attention is the idea of using a previous tier positioning for the initial placement of Hieroglyphic fragments. The main weakness of this approach is that the Hieroglyphic fragments then rely on at least one other tier, usually a Latin script tier, to be included in the interlinear text display even though users may sometimes desire to view the Hieroglyphic tier in isolation. However, this can be considered a less relevant limitation in the scope of this project, as we are specifically looking to generate good interlinear text in a web application. Furthermore, it can easily be overcome in future work by having the greedy algorithm as a fallback for when the user elects to view the Hieroglyphic tier in isolation.

So how would this approach work within our project? From the work on segmenting the Latin Tiers as described in Section 3.2.1, we have a series of lines of the other tiers with their Align Anchors included. At this stage, the anchors have already been aligned to each other if there is more than one other tier. For the Hieroglyphic tier, we can create new lines, place Align Anchors directly above their counterparts in the existing tiers and place all of the glyphs after said Align Anchor and before the next Align Anchor right after each Align Anchor on the same line. In an ideal scenario, none of the Hieroglyphic tier lines would be overflowing the container, and we could move on to the final step of the process. However, this is not always the case, and so we need to have a system in place to deal with overflow.

Before we move on to that however, we should also consider the occasional possibility that not all Hieroglyphs will fall between two Align Anchors. A good example of this would be the text *Ineni, biography, describing the southern gate in Karnak* from the St Andrews corpus, where we have a large chunk of text preceding the first Align Anchor where none of the tiers have any anchors. In this case, the start of the text acts as a sort of implicit anchor. This would cause an initially massive first line, but through methods we shall shortly explain, all of this content will be distributed across the available lines.

We elected to have a sort of two-pronged approach towards breaking the hieroglyphs based on how severe the overflow is. When the inserted hieroglyphs take up two or more lines as seen in this excerpt from *Dispute of a man with his ba*, we elect to insert a new line

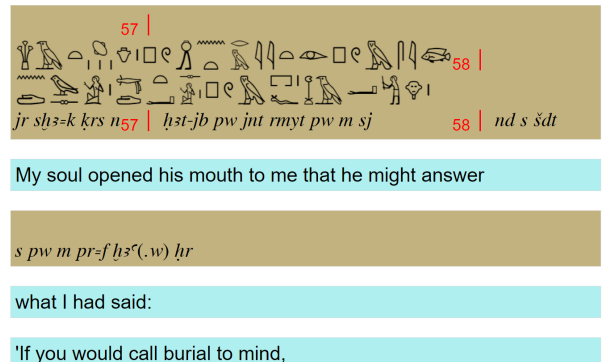


Figure 3.5: Excessive Overflow

of the hieroglyphic tier and slot the entirety of the overflowing fragment into it, i.e. in this case all of the glyphs after Align Anchor #58. The content in the other tiers with corresponding anchors, if any, will also be moved with it. This ties in with our iterative approach towards solving overflow, since further manipulation can be carried out once the current line's form is finalised.

When the overflow is limited to a single line, as seen in this excerpt from *Shipwrecked Sailor*, we incrementally remove glyphs from the end of the fragment until it fits into the line. Obviously, we can only step to valid breaking points as previously explained, and if none are found that are enough to end the overflow, the entire fragment will get removed, along with any anchored content from other tiers. The removed glyphs are then prepended to the next line which may be induced into an overflow by this, continuing the cycle.

In cases where only glyphs are broken off from a fragment without any Align Anchors, an obvious concern is that content from the other tiers, particularly the transliteration, will be visibly bunched up with the upper line instead of moving down with the glyphs. However in practice, this is generally not the case. Firstly, when the hieroglyphics overflow a line partially, it is often likely that the corresponding fragments in the other tiers would also have overflowed, particularly if the scaling ratio between both is close to 1, resulting in a proportionate amount of content from the other tiers already having been moved down. In fact, the opposite problem is more likely after this first stage of the hieroglyphic tier breaking, where you have empty “slots” where there is a transliteration and translation tier content, but no hieroglyphic content as shown in Figure 3.7. In order to deal with this issue, we have a second and distinct hieroglyphic tier

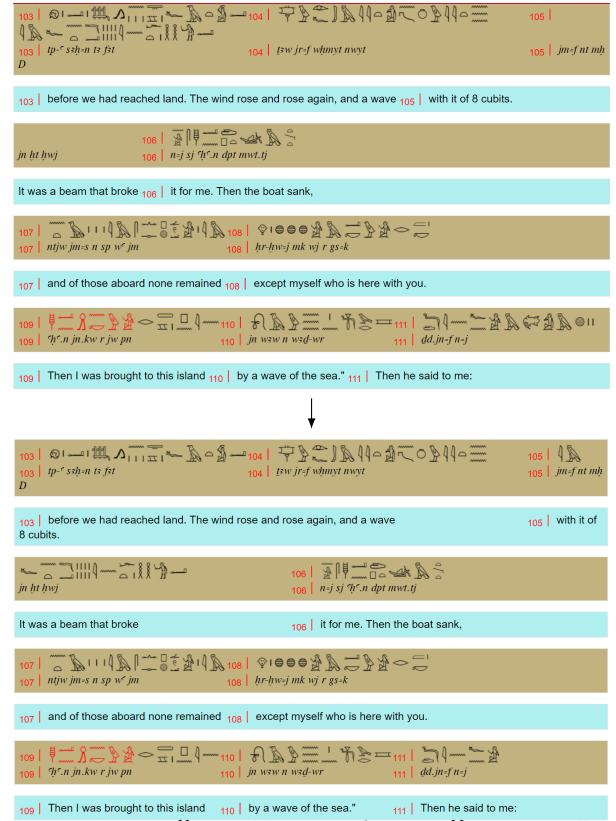


Figure 3.6: Limited Overflow Resolution

breaking procedure referred to as a Gap Filling algorithm to break certain Hieroglyphs even when they are not overflowing specifically to distribute them evenly alongside the other tiers.

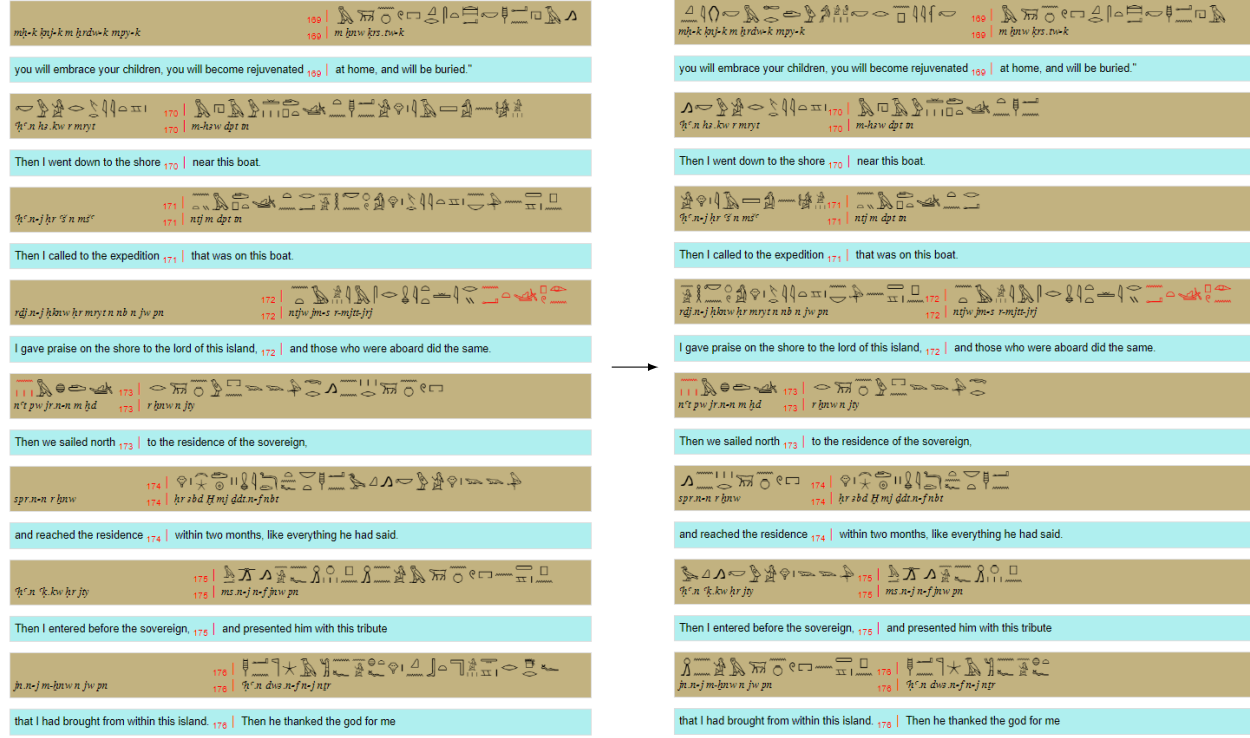
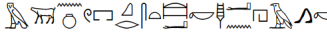
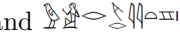
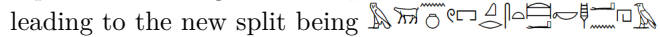
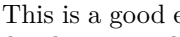


Figure 3.7: Hieroglyphic Redistribution

This works by detecting such gaps above the other tiers and then determining an appropriate width's worth of glyphs to fill them with. If there is already a gap due to the presence of “orphaned” transliteration and translation tiers, we can target to fill that gap with fragments to its full width *only if that gap is smaller than the overall fragment*. If the gap is bigger, however, or if there is an entire empty Hieroglyphic tier line, we have elected to break off a maximum of half the original fragment. Unlike our previous step, rather than trying to break off as little of the fragment as possible, in this case we are trying to break off as much as possible to fill the next line to our pre-determine maximum width. So we work forwards this time, iterating through all of the valid breaking points and breaking it off immediately if one that does not exceed the maximum

width is found. This results in a significant aesthetic improvement as seen in Figure 3.7. Notably, not only are the so-called “empty slots” improved, but even fragments split up in previous steps are augmented. If we take a closer look at the excerpt, we can see that the fragment after Align Anchor #169 was initially split into  and . Since we still had some space to spare before Align Anchor #170, we elected to break off  to fill the gap, leading to the new split being  followed by a line break followed by . This is a good example of how the Gap Filling Algorithm which was written with a bigger problem scope in mind also has useful benefits at a smaller scale. Notably, since we do not have any Align Anchors in-between #169 and #170, either split is equally valid given that our application is not meant to incorporate any information about the semantic relationships between the tiers, and thus the second option for splitting can be judged to be better from a purely aesthetic viewpoint. Within the Implementation section, the use of Gap Filling for the Latin script tiers will be explained.

3.3 Ordering

Choosing the order in which to process the tiers is a critical decision in the design of the application. The order in which the tiers are processed will affect the final appearance of the interlinear text, and so it is important to choose an order that is aesthetically optimal while balancing the requirements of the technology being used. In our case, the decision to handle the Latin tiers first was partly informed by pragmatism, since they could more easily be processed on the faster backend first and inserted into the view sent to the browser, while the hieroglyphic fragments required more complex operations on the front-end for alignment and breaking. It should be noted however, that this poses challenges when it comes to extending the application to be more general and able to accept an arbitrary number of tiers, since we cannot hope to establish some sort of hierarchy of every tier possible, especially when languages and writing systems we have never encountered before come into play. Nevertheless, our chosen order does have is that while the initial alignment is between the transliteration and translation tiers, since the final alignment is between the Hieroglyphics and the transliteration, those two tiers can always be expected to be aligned, while there is sometimes the possibility of the translation tier not being aligned to them.

4 Implementation

4.1 Technologies

The web application was built using the Node.js runtime environment, the Jade templating engine for designing the views, and the Express.js framework for handling the routing of the application. The RES Concatenation library by Nederhof provided a wide range of functionality for driving the rendering of hieroglyphics in HTML Canvases using client-side JavaScript. Another library by Nederhof called HieroJax allows for the conversion of the HTML Canvases into Unicode-based SVGs once the segmentation is complete.

4.2 Navigation

Moving throughout the application is fairly straightforward. The root of the website is a simple splash screen, from where we can access different index pages using a ribbon menu, by default a simple list of all the texts, but with options for navigating based on the book, kings or museums each text is classified under. Clicking on a particular text's link will take us to a dynamically generated view for the interlinear display that can be bookmarked by the user. The URL for each text is unique, so that the user can share it with others.

4.3 Application Stack

The backend of the application allows for faster loading and preparation of resources that do not require constant feedback based off the rendered web page. This includes in our case the Latin script tiers, but potentially would be applicable for any tier comprising of just Unicode text such as Hindi or Korean without any special formatting such as HTML Canvases. When the user loads a particular text, a kind of "ping-pong" takes place where a placeholder view is briefly loaded solely for checking the column width available the lines of each tier. This page then redirects the user to a modified path that includes this column width, which renders the actual view for the interlinear text with the contents of each line passed as JSON parameters to the Jade template. These lines are obtained by querying a backend API with the column width and the tier resource file path, which then carries out the segmentation and breaking outlined in the Design section.

This means that the somewhat slower frontend only has to deal with operations that are absolutely necessary. Since the fragments in the Hieroglyphic tier comprise of HTML Canvas objects, their breaking can only be handled by client-side JavaScript which has to respond to the overflow as it happens during the render in the browser window.

4.4 Customizability

As mentioned previously in the Literature Review, one concern we grappled with throughout the implementation of this web application was the degree to which the users should be given freedom to adjust the various parameters of the interlinear text display. The nature of a website in contrast to a desktop application is a greater expectation for some level of uniformity in the delivery of webpages. A user generally does not browse onto Wikipedia or the BBC news and expect to be given a dashboard to tinker with every aspect of the layout, font size or family. On the contrary, many of these aspects are hardcoded into the website's CSS as standard procedure, and where there is variation between clients, this largely arises from a more general category such as “serif” or “monospace” having been written in as opposed to a specific font family, a practice which in modern webpages is often relegated to the last resort option for when the website creator's original selection of fonts is not available on the client's system.

With this perspective, while it was certainly possible to offer fontface customization, it ultimately was not a priority for the project, which was conducted with limited time available. The benefits obtained in the scope of the main objectives was not proportionate to the effort that would have been involved. That is not to say that customization has been completely written off in this web application. On the contrary, the user is allowed to choose a custom font size for the Latin script tiers, as well as a custom scaling ratio for the Hieroglyphics. This is set by default at 20px and 1.5x, the latter being encouraged since hieroglyphs at scale with their Latin script counterparts suffer from reduced readability. Users are allowed to adjust either the font size or the scaling ratio at any time, and the application will reload with the new parameters. These settings are stored locally, which means they persist as the user navigates through the corpus onto other texts.

4.5 Caching

Due to the numerous repeated processes involved in producing an acceptably wrapped and aligned interlinear text display, the time taken to finish loading a text can be high, particularly when the column width is lowered and the font size is increased. To mitigate this, we have implemented a caching system that stores the results of the text processing in the user's local storage, i.e. on a cookie. It is quite straightforward to achieve this, as we can simply write the inner HTML of the table that contains the tiers with a unique identifier as the key. This unique identifier consists of the name of the text, the column width, the font size and the scaling ratio. When the user navigates to a text that they

have already viewed with the current parameters, this inner HTML is found and loaded into the webpage, saving the user a significant amount of time. The only further processing that will have to occur is the drawing of the Canvas objects containing the hieroglyphic fragments, since this cannot be trivially serialised. However, just from the RES of each fragment, we are able to restore the original appearance.

4.6 Anchor Alignment

It stands to reason that for n number of tiers, we would have nC_2 different pairs of tiers to align. Our current implementation works with three tiers, so the number of alignments needed is not too high. Nevertheless, alignment is not straightforward and it has to be handled according to the order we determined previously in pairs.

We first align the Latin tiers such as transliteration and translation. Each align anchor is represented as a bright red sub-scripted number with a vertical line, held within a HTML span element. This allows us to use the Bounding Client Rectangle of the span to obtain the absolute co-ordinates of each anchor on the webpage. We can see which anchor is further behind on the x-axis and provide that one with a left margin in the CSS styling amounting to the pixel width difference so that is vertically aligned. This is simple and also automatically pushes all of the text after it, allowing the process to be quite iterative. Overflow does not occur because the previous segmentation already took the extra space that would be taken up into account.

The alignment for the non-Latin tiers like the Hieroglyphs is quite similar at the basic level. The main difference, however, is that since overflow is quite possible, we have to constantly loop back to the previous step of breaking the Hieroglyphs into fragments and re-assigning them to lines, before checking the alignment again. Since the transcription and translation tiers are already aligned, we only need to compare the hieroglyphic tier's anchor to one of them. We create a custom HTML element called `<hil>` to store a single line of the hieroglyphic line. This allows us to leverage the CSS border boxes of the hil element by counting the number of them using the `getClientRects` method. That corresponds to the number of lines that the `<hil>` element is comprised of, so until it comes down to one we know that we have to keep breaking the line to prevent overflow. We also check if the bounding `<hil>` element is longer than the width of the table row, since in some cases, particularly if a single fragment is excessively long, the hieroglyphic line will spill out of the table rather than wrapping into multiple lines.

4.7 RES Insertion

Each of the XML indexes for an interlinear text comes with a file reference to the different resources corresponding to each tier. For the hieroglyphic tier, this typically appears as a reference to a XML file consisting of the RES of each fragment, most of which are "assigned" to anchor points, but with some exceptions such as the preamble of *Ineni*, *biography*, *describing the southern gate in Karnak*. Since our design decisions within the project have led to the Latin script tiers being rendered first, we have a view with empty slots waiting for the RES fragments as seen here. It is crucial to note that at this stage, we do not align the anchors, since obviously such alignment would be meaningless without the actual rendered width of hieroglyphs. Hence, fragments are just sequentially assigned to lines that contain their anchor points. In cases without anchor points such as *Ineni*, they are just appended to the earliest possible line based on their order in the source file.



Figure 4.1: Empty Slots

In contrast to the data retrieval for the Latin script tiers, which was carried out on the backend, the insertion of RES fragments is done with a XML HTTP Request on the client side, where the fragments are parsed and assigned to their correct slots, with footnotes getting filtered out and displayed at the bottom correctly.

4.8 Conversion from RES to Hieroglyphics

We use HTML Canvas representations as an intermediate representation of the Hieroglyphics fragments. This allows us to obtain a simulation of the actual width taken up by the hieroglyphic fragments after anchor alignment which allows for detection and handling of overflows.

The Canvas is obtained using the ResWeb methods in the RES Concatenation library. This works by parsing the text content of the canvases, which is the RES, and using a stack to keep track of the glyphs that need to be drawn as well as the many special operations such as ligatures that need to be conducted between them. The hieroglyphs are drawn using a look-up table that assigns the RES codes to their Unicode counterparts, and renders them using the slightly customized New Gardiner font. The font size is derived by multiplying

the Latin script tiers’ font size with the scaling ratio and assigning that value to the font size attribute of the canvas objects, which the library can then refer to. It should be noted that the final result is rasterized which leads to a lacking in sharpness or selectability.

4.9 Conversion from Canvas to SVG

The application was extended to allow for either an automatic conversion of the representation of the Egyptian Hieroglyphics from Canvas objects to Unicode-SVG.

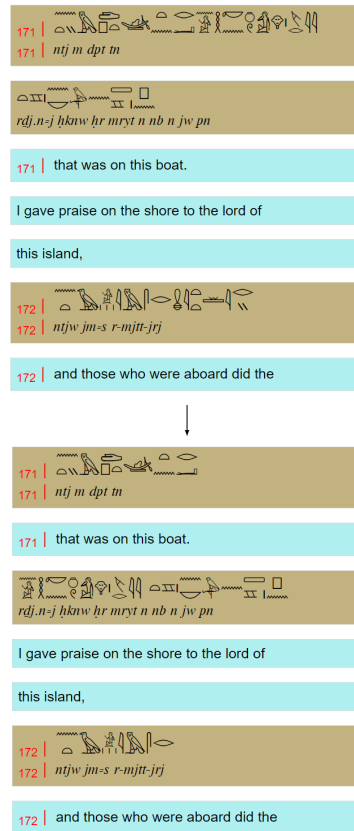


Figure 4.2: Translation Tier Gap Filling

This could be done once the breaking and alignment is finalised, since the RES is preserved within the HTML DOM structure. Since the main advantage of the Canvas representation is during internal manipulation, this would improve the user experience by allowing them to be able to select and copy the Hieroglyphic text, a property that would be preserved even when the webpage is printed into a PDF.

A crucial challenge encountered while attempting to convert the hieroglyphic fragments from Canvases to SVGs was that the RES Concatenation library (for Canvases) and the HieroJax library (for SVGs) had variable naming clashes which prevented both from being loaded as source scripts by the same webpage. Instead, the approach we took was to create a local instance of the HieroJax RES Automatic Converter². We were then able to use the JavaScript Window interface to access this tool in a “headless mode” to input the RES of each canvas fragment, obtain a Unicode-SVG representation whose HTML could just be swapped into where the Canvas used to be. With local hosting, the HieroJax instance is able to ac-

²Public version available at <https://nederhof.github.io/hierojax/resconversion.html> by Nederhof

4.10 Gap Filling

4.11 Printing

Here for example, we can see a sample inset from a PDF printout of *Stone naos of Amenmose*. Note that thanks to the SVG conversion of the Hieroglyphics, if this paper is being read electronically, you will be able to select and copy everything in the printout including the hieroglyphics.



5 Evaluation

The primary objective has been achieved to a satisfactory degree, with the web application being able to display interlinear text in a manner that is both aesthetically pleasing and semantically accurate. The application is able to handle the majority of the texts in the St Andrews corpus, and the user is able to adjust the font size and scaling ratio to their liking. The application also includes some quality-of-life features such as caching and printing which contribute towards user friendliness. The secondary and tertiary objectives largely remain to be achieved.

Multiple texts within the corpus have been tested across a variety of parameters to ensure their conformance to expected standards of alignment and semantic coherence. This iterative process of testing allowed for the identification of a number of bugs and edge cases that were not initially considered, and the subsequent fixing of these issues. It was difficult to test the application on different devices such as smartphones, as it was a locally hosted instance, however modern browsers did allow for a degree of emulation of these contexts using their developer tools, which proved to be without issue.



Figure 5.1: Scalability

6 Future Work

There are a number of features that remain to be implemented to achieve full parity in scope with the original application. This includes full dexterity for dealing with additional tiers such as non-English translations, or differing versions of Hieroglyphic transcription such as seen in texts like *Eloquent Peasant*.

Another area that could be extended is the use of supplementary information in the text production. This includes images of some of the hieroglyphs in their original appearance as a visual reference for the user, as well as the use of the supplementary precedence constraints available with a number of the texts, as opposed to solely relying on the vertical anchor links.

A significant addition would be support for Orthographic tiers such as those seen in *Westcar Papyrus* or *Shipwrecked Sailor* which provide graphical aids to explain the derivation of the transliteration from the glyphs. This would require significant work done to create a driver of sorts to visualize the orthographic relationship based on the XML data into perhaps a SVG. Such an orthographic tier would also have fresh concerns for alignment and breaking in particular, due to certain constraints being much more stricter such as not being able to break a single graphic into segments.

Another feature that entails a large amount of effort would be the full editing functionality envisioned in the Secondary objectives. This would require substantial additions to the current interface to provide for this, including the ability to add, remove or modify align anchors. We would need to implement a robust method for reliably converting edits made within the web application's display context to the original XML or plain-text data storage methods. This would also require a significant amount of testing to ensure that the changes made are accurately persisted in the final output. The user experience for editing the Hieroglyph tier is also of particular concern, since the current SVG display is not particularly interactive and may require a more robust understanding within the application of the relationship between the RES and individual glyphs and the use of drop-down menus or other tools to decrease the learning curve involved in editing that tier.

7 Conclusion

The PhilologEg-Web application successfully demonstrates the feasibility of displaying interlinear Ancient Egyptian hieroglyphic texts on the web. By leveraging modern web technologies like Node.js, Express, and JavaScript, the application overcomes challenges around rendering variable-width hieroglyphic writing systems alongside linear Latin script tiers. Through a multi-step process of segmenting and aligning tiers, breaking overflowing fragments, and redistributing content, the application achieves a satisfactory interlinear representation adaptable to different screen sizes and user preferences.

While meeting the primary objective, the application's current scope is limited, lacking features like unlimited tiers, orthographic visualization, and full editing capabilities. However, the modular design provides a strong foundation for extending functionality in future work. Potential areas of improvement include support for additional languages, incorporating supplementary information like images and precedence constraints, and developing robust editing tools tailored to the unique aspects of hieroglyphic scripts.

Overall, this project represents a significant step towards bringing the study and analysis of Ancient Egyptian texts into the digital age, enabling wider accessibility and collaboration within the field of Egyptology. By adapting prior work on hieroglyph encoding and rendering for the web environment, this application paves the way for further innovation in the online representation and manipulation of non-linear writing systems, benefiting not only Egyptologists but scholars across diverse philological domains.

8 References

8.1 Research Literature

1. Nederhof , M J 2009 , **Automatic creation of interlinear text for philological purposes** , *Traitement Automatique des Langues* , vol. 50 , no. 2 , pp. 237-255 .
2. Schroeter, Ronald, and Nicholas Thieberger. **EOPAS, the EthnoER online representation of interlinear text.** *Sustainable Data from Digital Fieldwork. Proceedings of the conference held at the University of Sydney*, 4-6 December 2006. Sydney University Press, 2006.
3. Bow, Cathy and Hughes, Baden and Bird, Steven. (2003). **Towards a general model of interlinear text.**
4. Nederhof, Mark Jan. **Automatic alignment of hieroglyphs and transliteration.** *Information Technology and Egyptology in 2008* (2009).
5. Wildish, Mark (2012) **Hieroglyphic Semantics in Late Antiquity.** Doctoral thesis, Durham University.

8.2 Egyptological Texts

1. Goedicke, H. A. N. S., Johns Hopkins Press, and R. Hannig. **Dispute of a man with his ba.** *Papyrus Berlin 3024*.
2. A.M. Blackman. **Shipwrecked Sailor** *Middle-Egyptian Stories – Part I.* Fondation Égyptologique Reine Élisabeth, Brussels, 1932.